

Tender Analyzer — Build Specification

שם החברה: חברת בדיקה בע"מ

You are implementing **Tender Analyzer**, an AI-powered web application that helps Israeli businesses respond to government tenders (מכרזים) faster. The MVP targets one specific tender — Israel Police tender **29/2026** for building maintenance services in the Northern district — but the architecture must generalize to other Israeli government tenders later.

The user is an Israeli web developer with a Hebrew/English mixed workflow, working from `mly.co.il`. The app's UI is in Hebrew. Documents and extracted content are in Hebrew. Code, comments, and commit messages are in English.

1. Goals & success criteria

What the app does

A bidder uploads a tender package (a 240-page main PDF with 11 annexes (נספחים), plus an optional Excel reference file). The same data points (company name, registration number, signatory ID, etc.) repeat across many forms inside the package. Filling them manually takes days. The app:

1. **Reads** the tender package — extracts text from PDFs and structure from Excel
2. **Analyzes** with Claude — identifies every fillable field across every annex, finds key dates, identifies the price/scope sections
3. **Deduplicates** — recognizes that "שם החברה", "שם המציע", and "שם התאגיד" all map to one canonical "company name" data point
4. **Asks the user** — a wizard-style questionnaire collects each unique data point once
5. **Improves narrative answers** — for long-text fields (methodology, work plan), the user writes freely and can request an AI improvement (uses OpenAI)
6. **Fills the original PDF** — writes user's answers onto the **original PDF** at exact coordinates using **PyMuPDF** (Python). The output PDF must remain byte-compatible with the original (any AI verification system on the police side must see the same document with text added on top, NOT a regenerated facsimile). PyMuPDF is chosen because it has native Hebrew RTL support via HarfBuzz — `pdf-lib` does not, and would require manual bidi handling that wastes 1-2 weeks.

7. **Produces a checklist** — milestones, dates, and physical actions the user must still do manually (signatures, סיור קבלנים, ערבות בנקאית, physical submission).

Success criterion for the MVP

Given the police tender 29/2026 PDF and Excel files (provided in `samples/`), the app must:

- Extract correctly all 11 annexes and identify their fillable fields
 - Produce a deduplicated questionnaire of at most ~30 questions
 - Generate filled PDFs that look identical to the originals with user's data inserted at the right places
 - Display a working timeline of all key dates from the tender
 - Allow a user to start filling, log out, log back in days later, and resume from the exact same step
-

2. Stack (non-negotiable)

Main app (Node)

- **Framework:** Next.js 15 with App Router, TypeScript strict mode
- **Database:** PostgreSQL 16 via Prisma ORM
- **Auth:** NextAuth (Auth.js) v5, Credentials provider, email + password, bcrypt for hashing. Multi-user. No organizations.
- **UI:** Tailwind CSS + shadcn/ui components, configured for RTL
- **Hebrew font:** Heebo (SIL OFL, free) — bundle .ttf files in `public/fonts/` for the web UI
- **Icons:** `lucide-react`
- **Forms:** React Hook Form + Zod
- **AI extraction (structured):** `@anthropic-ai/sdk` — Claude Sonnet 4.7 (`claude-sonnet-4-5` or latest available)
- **AI narrative improvement (creative):** `openai` SDK — GPT-4o-mini for cost
- **PDF text extraction:** `pdf-parse`
- **PDF page-to-image (for vision-based field detection):** `pdfjs-dist` + `canvas`

- **Excel reading:** `exceljs`
- **mr.gov.il scraper:** native `fetch` + `cheerio`
- **Job queue:** `pg-boss` (PostgreSQL-backed, no Redis dependency)
- **Logging:** `pino`
- **Validation:** `zod` everywhere

PDF overlay microservice (Python)

A separate FastAPI service in the same repo that handles ALL PDF writing. Reason: PyMuPDF has native Hebrew RTL via HarfBuzz; the JS ecosystem doesn't have an equivalent. Isolating PDF overlay in Python lets us keep the JS app simple while using the right tool for Hebrew.

- **Runtime:** Python 3.12
- **Framework:** FastAPI + Uvicorn
- **PDF library:** `PyMuPDF` (license: AGPL-3 — acceptable for v1; if commercializing as SaaS later, evaluate Artifex commercial license)
- **Validation:** Pydantic v2
- **Logging:** structlog
- **Communication with Next.js:** internal HTTP via Docker Compose network
(`http://pdf-service:8000`)

Deployment

Hetzner VPS, Docker Compose with services: `app` (Next.js), `pdf-service` (Python FastAPI), `postgres`, `caddy` (reverse proxy + auto Let's Encrypt). Single VPS, one `docker-compose.yml`.

Do NOT add additional dependencies without strong justification. Especially do NOT add: a state management library (use React state + server actions), a CSS-in-JS library (use Tailwind), Redux/Zustand, or alternative ORMs.

3. High-level architecture

Browser (React, RTL Hebrew UI, mobile-responsive)

|

▼

```

Next.js App Router (server components + server actions + route handlers)
|
├─ Anthropic API    - structured field extraction
├─ OpenAI API       - narrative improvement
├─ PostgreSQL via Prisma - user data, tender data, answers, progress
├─ Local filesystem `/data/tenders/{tenderId}/`
|   ├─ original.pdf
|   ├─ original.xlsx (if any)
|   ├─ extracted/{annexId}.json      (fields + coordinates per annex)
|   ├─ output/{annexId}.pdf         (filled output PDFs)
|   └─ output/all.zip               (bundle for download)
├─ pg-boss job queue - long-running extraction & PDF generation
├─ mr.gov.il        - cheerio scraper for URL-based imports
|
└─ pdf-service (Python FastAPI, internal Docker network)
    └─ PyMuPDF - fills original PDFs with user data, native Hebrew RTL

```

All long-running operations (PDF text extraction, Claude analysis, OpenAI narrative improvement, PDF overlay generation) run as background jobs via `pg-boss`. API routes return immediately with a job ID. The frontend polls `/api/jobs/[jobId]` every 2 seconds for status, OR uses Server-Sent Events for real-time updates (preferred — implement with a simple `/api/jobs/[jobId]/stream` SSE endpoint).

The Next.js app calls the Python PDF service via internal HTTP. The Python service is stateless: it receives `{pdf_bytes, fields[]}`, returns filled PDF bytes. No database access. No file system writes. The Next.js app is responsible for all storage.

The user can navigate away from a processing tender and return later — progress is persisted in the DB.

4. Data model (Prisma schema)

```

// prisma/schema.prisma

generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

```

```

model User {
    id          String      @id @default(cuid())
    email       String      @unique
    passwordHash String
    name        String?
    createdAt   DateTime    @default(now())
    tenders     Tender[]
}

model Tender {
    id          String      @id @default(cuid())
    userId      String
    user        User        @relation(fields: [userId], references: [id], onDelete:

    sourceUrl   String?      // mr.gov.il URL if applicable
    title       String
    publisher   String?      // e.g. "מחשבת ישראל"
    tenderNumber String?     // e.g. "29/2026"
    deadline    DateTime?    // submission deadline
    ervutAmount Decimal?     // ערבות הצעה amount in NIS
    contactName String?
    contactEmail String?
    contactPhone String?
    submissionMethod String? // text description

    status      TenderStatus @default(UPLOADED)
    // UPLOADED | EXTRACTING | READY | FILLING | GENERATED | ERROR

    rawSummary   Json?      // top-level Claude-extracted summary
    errorMessage String?

    createdAt    DateTime @default(now())
    updatedAt    DateTime @updatedAt

    files        TenderFile[]
    annexes     Annex[]
    questions    Question[]
    answers      Answer[]
    milestones   Milestone[]
    jobs         Job[]
}

enum TenderStatus {
    UPLOADED
    EXTRACTING
    READY

```

```

    FILLING
    GENERATED
    ERROR
}

model TenderFile {
    id          String    @id @default(cuid())
    tenderId    String
    tender      Tender    @relation(fields: [tenderId], references: [id], onDelete: C
    filename    String
    fileType    String    // "pdf" | "xlsx" | "docx" | "zip"
    filePath    String    // relative to /data
    pageCount   Int?
    isMain      Boolean   @default(false)
    createdAt   DateTime  @default(now())
}

// Each annex (אָנעקס) is a logical sub-document inside the main PDF or a separate
model Annex {
    id          String    @id @default(cuid())
    tenderId    String
    tender      Tender    @relation(fields: [tenderId], references: [id], onDelete
    code        String    // e.g. "ה", "ב'", "א'2"
    title       String    // e.g. "תצהיר בדבר אי תיאום מכרז"
    fileId      String?   // which TenderFile it lives in (usually the main PDF)
    startPage   Int?     // page number in the source PDF (1-indexed)
    endPage     Int?
    status      AnnexStatus @default(PENDING)
    // PENDING | EXTRACTED | FILLED | HUMAN_ONLY
    fields      Field[]
    createdAt   DateTime  @default(now())
}

enum AnnexStatus {
    PENDING
    EXTRACTED
    FILLED
    HUMAN_ONLY
}

// A fillable field detected in an annex
model Field {
    id          String    @id @default(cuid())
    annexId    String
    annex      Annex     @relation(fields: [annexId], references: [id], onDelete:
    label       String    // raw label as it appears in the PDF
    fieldType   FieldType

```

```

page          Int          // page in the source PDF
bbox          Json         // { x: number, y: number, width: number, height: number
required      Boolean      @default(true)
maxLength     Int?
options       Json?        // for select/checkbox: list of options
questionId    String?      // mapped (deduplicated) question
question      Question?    @relation(fields: [questionId], references: [id])
rawConstraints Json?       // validation rules extracted from the tender
}

```

```

enum FieldType {
    SHORT_TEXT      // company name, contact name
    LONG_TEXT       // narrative answers (methodology, etc.)
    NUMBER
    CURRENCY        // NIS amounts
    DATE
    CHECKBOX
    RADIO
    SELECT
    ID_NUMBER       // ת.ז. (9 digits, validate)
    COMPANY_ID      // ע.מ. (9 digits, validate)
    PHONE
    EMAIL
    ADDRESS
    SIGNATURE       // human-only
    STAMP           // human-only
    LAWYER_BLOCK    // human-only (separate signatory)
}

```

// Deduplicated question – one question may map to many fields across annexes

```

model Question {
    id          String      @id @default(cuid())
    tenderId    String
    tender      Tender      @relation(fields: [tenderId], references: [id], onDelete:
    category    QuestionCategory
    label       String      // Hebrew label shown to user
    helpText    String?     // extra context shown under label
    fieldType   FieldType   // dictates UI input type
    required    Boolean      @default(true)
    options     Json?
    validation  Json?        // zod-compatible validation hints
    order       Int         // display order within category
    appearsInAnnexes Json    // ["1" ,"1a" ,"2'a"] – shown to user as helper to
    fields      Field[]
    answer      Answer?
}

```

```

enum QuestionCategory {
    COMPANY_INFO          // step 1
    SIGNATORY_INFO        // step 2
    EXPERIENCE             // step 3
    DECLARATIONS           // step 4 (yes/no with optional follow-ups)
    NARRATIVE              // step 5 (long text + AI improvement)
    REVIEW                 // step 6 (final review, no inputs)
}

model Answer {
    id          String    @id @default(cuid())
    tenderId    String
    tender      Tender    @relation(fields: [tenderId], references: [id], onDelete:
    questionId  String    @unique
    question    Question  @relation(fields: [questionId], references: [id], onDelete:
    value       Json      // string, number, boolean, or structured value
    aiImproved  Boolean    @default(false)
    originalValue Json?   // before AI improvement, so user can revert
    updatedAt   DateTime  @updatedAt
}

model Milestone {
    id          String    @id @default(cuid())
    tenderId    String
    tender      Tender    @relation(fields: [tenderId], references: [id], onDelete:
    title       String    // "הגשת שאלות הבהרה", "הגשת טופס", "סיור קבלנים"
    description  String?
    date        DateTime
    category    MilestoneCategory
    isDeadline  Boolean    @default(false) // true for the final submission deadline
    done        Boolean    @default(false)
    doneAt      DateTime?
    order       Int
}

enum MilestoneCategory {
    DATE          // time-anchored event
    DOCUMENT      // a document needed (אישור ניהול ספרים, etc.)
    GUARANTEE     // ערבות בנקאית
    SIGNATURE     // signatures needed
    PHYSICAL_ACTION // סיור קבלנים, hand delivery, etc.
}

// Background job tracking
model Job {
    id          String    @id @default(cuid())
    tenderId    String?

```

```

    tender      Tender?  @relation(fields: [tenderId], references: [id], onDelete:
type          String    // "extract", "analyze", "generate"
status        JobStatus @default(PENDING)
progress      Int        @default(0)
progressMessage String?  // "Reading page 47 of 240..."
result        Json?
error         String?
startedAt     DateTime?
finishedAt    DateTime?
createdAt     DateTime @default(now())
}

enum JobStatus {
    PENDING
    RUNNING
    DONE
    FAILED
}

// Saved progress for resuming wizard
model WizardProgress {
    id          String    @id @default(cuid())
    tenderId    String    @unique
    currentStep Int        @default(1)
    updatedAt   DateTime @updatedAt
}

```

5. UI structure

The UI is **Hebrew-only** with `dir="rtl"` set at the root. Visual style: clean and modern, similar to the Stripe dashboard — white background, generous whitespace, subtle shadows on cards, rounded corners, primary blue (`#0570DE`), status colors (green = done, amber = in-progress, gray = pending, red = blocking/overdue). Use `shadcn/ui` components configured for RTL. Mobile-first responsive: works on phone, tablet, and desktop.

Routes

<code>/</code>	– Landing → redirect to <code>/dashboard</code> if logged in,
<code>/login</code>	– Email + password
<code>/register</code>	– Email + password
<code>/dashboard</code>	– List of user's tenders
<code>/tenders/new</code>	– Modal/page for creating a new tender (upload P
<code>/tenders/[id]</code>	– Tender overview (3 tabs: Summary, Forms, Timel

<code>/tenders/[id]/wizard</code>	- Wizard-style questionnaire
<code>/tenders/[id]/wizard/[step]</code>	- Specific wizard step (1-6)
<code>/tenders/[id]/output</code>	- Generate/download filled PDFs
<code>/api/...</code>	- Route handlers as specified in section 8

Screen 1 — Login / Register

Standard NextAuth Credentials UI. Hebrew labels:

- "הרשמה" / "התחברות"
- Email + password fields, validation messages in Hebrew
- "אין לך חשבון? הרשמה" link
- Logo at top: "מנתח המכרזים" (placeholder logo, just typography for v1)
- Mobile-friendly, single column

Screen 2 — Dashboard (tender list)

Top bar: logo on right (RTL), user menu on left with "התנתק" option. Page heading: "מכרז חדש +"

Tender cards (or table on desktop):

- Title (large)
- Publisher + tender number
- Deadline countdown ("עוד 13 ימים", red if < 48 hours)
- Progress bar: "12/47 הושלמו"
- Status badge: "שגיאה" / "מוכן להורדה" / "ממולא חלקית" / "בעיבוד..." / "מוכן למילוי"
- Click anywhere → `/tenders/[id]`

Empty state: friendly illustration (use a placeholder lucide icon) + "טרם הוספת מכרזים. בוא" + "נתחיל" button.

Screen 3 — New tender modal

Two tabs: "קישור מ" / "העלאת קובץ" - `mr.gov.il`

Tab 1: react-dropzone, accepts `.pdf` and `.xlsx`, multi-file, max total 50MB, shows file list with size and remove button. Tab 2: text input + "מצא מכרז" button. Validates URL is from `mr.gov.il`. On click, scrapes metadata server-side and shows a preview (title, publisher,

deadline) before confirming.

Submit: "התחל ניתוח". Closes modal, navigates to /tenders/[id], shows banner "המכרז...".

Screen 4 — Tender overview

Top section (always visible above tabs):

- Tender title, publisher, tender number
- Deadline countdown (LARGE if < 48 hours, red color)
- Progress: "12/47 שדות הושלמו" with progress bar
- Primary CTA: "המשך מילוי" (jumps to wizard at the user's last step) OR "התחל מילוי" if not started

Tabs:

סיכום (Summary) Card grid:

- "על המכרז" — short Claude-generated summary (2-3 paragraphs)
- "תנאי סף" — bulleted list
- "היקף העבודה" — short scope description (for the police tender, includes # of facilities, total m², equipment counts pulled from the Excel)
- "אופן הגשה" — submission method
- "ערבות הצעה" — amount + validity period
- "ביטוח נדרש" — required insurance coverage

טפסים (Forms) List of annexes (one row per annex):

- Code (e.g., "נספח א'2") + title

• fields total / # fillable by AI / # human-only

- Status badge:  "ידיני בלבד"  / "דורש מילוי"  / "מוכן"
- Action buttons: "תצוגה מקדימה" (opens original PDF page in modal), "מלא" (jumps to relevant wizard step)

ציר זמן ופעולות (Timeline & Actions) Toggle at top: "ציר זמן" (default) / "רשימת פעולות"

Timeline view: vertical timeline with milestones.

- Each milestone: date (large), title, description, category icon, status, "סמן כבוצע" button
- Milestones in `MilestoneCategory.DATE` are time-anchored
- Other categories appear at their relevant dates if known, else grouped under "ללא תאריך" מוגדר"
- The final submission deadline is highlighted in red
- Past-due items pulse with a red border

Checklist view: grouped by category with collapsible sections. Same milestones, just grouped differently.

Both views write to the same data — toggling is purely visual.

Screen 5 — Wizard

URL: `/tenders/[id]/wizard/[step]` where step is 1-6.

Top: progress bar + step indicator: "שלב 3 מתוך 8: פרטי המציע" Layout: centered, max-width ~720px on desktop, full-width on mobile.

Step 1 — פרטי החברה (Company info)

- שם החברה
- ח.פ. (validate 9 digits)
- כתובת
- טלפון
- אימייל
- אתר אינטרנט (optional)

Step 2 — פרטי המורשה חתימה (Signatory info)

- שם מלא
- ת.ז. (validate 9 digits + checksum)
- תפקיד
- טלפון

Step 3 — ניסיון קודם (Experience)

- שנות ותק

- Repeating list: עבודות דומות שבוצעו בעבר (name, scope, year, contact)
- מחזור כספי שנתי

Step 4 – תצהירים (Declarations)

- Yes/no questions with optional "פרט" follow-up text fields
- E.g., "האם נחקרת בעבירה של תיאום מכרז?" → if yes, show text area for details

Step 5 – שאלות פתוחות (Narrative questions)

- Each long-text field gets its own card
- Text area, char count, max-length warning if specified by tender
- Below textarea: "🌟 AI שפר עם" button — calls OpenAI to rewrite the text more professionally
- Shows before/after, with "השתמש בגרסה זו" / "השאר את הטקסט שלי" / "ערוך עוד"

Step 6 – סקירה (Review)

- All answers shown in collapsible groups
- Edit button per group
- "צור מסמכים" button at bottom — creates the PDF generation job and navigates to /tenders/[id]/output

Each input has helper text below: "מופיע ב: נספח א'2, נספח ה', נספח ז" Auto-save: debounce 500ms on every change. Visual indicator: "נשמר" flashes in corner. Bottom: "חזור" / "המשך" buttons. "המשך" disabled until step is valid.

Screen 6 – Output / Generate

Header: "המסמכים שלך"

While generating: progress bar + message "ממלא נספח 4 מתוך 11"

When done:

- Top: "ZIP-הורד הכל כ" big primary button
- List of filled annexes:
 - Annex code + title
 - "תצוגה מקדימה" (renders filled PDF in modal/iframe)
 - "השווה למקור" toggle (side-by-side with original)

- "הורד" (single PDF)
- Bottom card: "מה עוד צריך לעשות?" — final reminder of remaining milestones (signatures, ערבות, סיוור קבלנים, physical submission)
- Email-to-self button is NOT implemented in v1 (commented out in code with `// TODO v1.5`)

Deadline notifications (in-app only for v1)

A small toast/banner appears on Dashboard and Tender overview when:

- Any milestone is < 48 hours away → amber
- Any milestone is < 24 hours away → red, persistent
- Any milestone is past due and not marked done → red, persistent

Use `shadcn/ui` `Sonner` or `Toast` component. NO email reminders in v1.

6. The AI extraction pipeline (this is the heart of the app)

This pipeline runs in a `pg-boss` job when a tender is uploaded. It has multiple sequential passes:

Pass 1 — File discovery and classification

For each uploaded file:

- If PDF: extract metadata via `pdf-parse`, get page count, sample first 5 pages of text
- If Excel: list sheets and shape via `exceljs`
- Send file inventory to Claude with prompt: *"For each file, classify as: MAIN_TENDER, ANNEX_FORM, SPEC, CONTRACT, REFERENCE_DATA, PRICE_SHEET. Return JSON."*
- Update DB with classifications

Pass 2 — Annex segmentation (for the main PDF)

The 240-page PDF contains all 11 annexes inline. Use Claude to find their page ranges:

- Extract full text via `pdf-parse`
- Send to Claude with prompt: *"This is a tender package. Find every annex (נספח) and return its code, title, start page, end page. Return JSON array."*

- Create `Annex` records in DB

Pass 3 — Tender metadata extraction

- Send the first 60 pages (the explanatory section) to Claude
- Prompt: *"Extract tender metadata as JSON: title, publisher, tenderNumber, deadline (ISO 8601), ervutAmount, contactName, contactEmail, contactPhone, submissionMethod, eligibilityCriteria (array), insuranceRequirements (object), evaluationCriteria (array of {criterion, weight}). Also extract every key date with title and date as Milestone[]."*
- Store on the `Tender` record + create `Milestone` records

Pass 4 — Field extraction per annex (the hard one)

For each annex:

- Render its pages to images using `pdfjs-dist` + `canvas` at 150 DPI
- Send the page images + text to Claude with vision input
- Prompt (truncated): *"Identify every fillable field in this annex. For each, return: label (Hebrew), fieldType (one of the FieldType enum values), page (1-indexed within the source PDF), bbox (in PDF points: x, y, width, height — origin is bottom-left of the page), required (bool), maxLength (if specified), and any constraints. Mark signature fields, stamp fields, and lawyer-attestation blocks as SIGNATURE/STAMP/LAWYER_BLOCK. Mark fields >100 words expected as LONG_TEXT. Return JSON array."*
- IMPORTANT: Claude vision returns coordinates in image pixel space. You MUST convert to PDF point space (1 point = 1/72 inch). Page dimensions are available from `pdfjs-dist (page.getViewPort({ scale: 1 }) gives pts)`. Conversion: `pdf_y = page_height_pts - (image_y / image_height_px) * page_height_pts` (PDF origin is bottom-left, image origin is top-left).
- Save `Field` records linked to the annex

Pass 5 — Field deduplication into questions

This is the step that makes the app valuable.

- Gather all `Field` records across all annexes for this tender
- Send the list to Claude with prompt: *"Here are all fillable fields across this tender's annexes. Group fields that ask for the same data. Output a list of canonical questions:*

each question has a category (COMPANY_INFO | SIGNATORY_INFO | EXPERIENCE | DECLARATIONS | NARRATIVE), label (Hebrew), helpText (optional), fieldType, required (bool), order within category, and a list of field IDs that map to it. A field with fieldType SIGNATURE/STAMP/LAWYER_BLOCK should NOT be mapped to any question (these are human-only). Return JSON."

- Create `Question` records, link `Field.questionId` to the appropriate question

Pass 6 — Generate scope summary

- For the police tender specifically, use the Excel to enrich the summary
- Read all sheets from the reference Excel via `exceljs`
- Send sheet structure to Claude: *"Summarize the scope: how many facilities, total square meters, what types of equipment/systems are listed across all sheets. Return concise Hebrew text."*
- Save to `Tender.rawSummary`

Error handling & resumability

- Each pass is its own `pg-boss` job
- If a pass fails, the user sees an error state on the tender with a "נסה שוב" button
- Partial progress is preserved — re-running pass 4 doesn't redo passes 1-3
- All Claude/OpenAI calls have retries with exponential backoff (3 attempts)
- Token cost tracking: log every API call's token usage to a `Job.result` field for cost monitoring

Prompt engineering rules

- Always request structured JSON with explicit schema
- Use Anthropic's `tool_use` feature OR strict response prefix (`{ " }`) to force JSON
- Validate every response against a Zod schema before saving to DB
- On validation failure, retry once with a "your previous response was invalid: [error]" follow-up
- All prompts must specifically request Hebrew text in fields where Hebrew is expected
- Never ask Claude to invent data — if a field can't be confidently identified, mark it as `OTHER` rather than guess

7. PDF overlay (writing answers back to the original PDF)

This runs as a `pg-boss` job in the Next.js app. The job orchestrates: gather field data → call Python pdf-service → save returned PDF bytes → bundle into ZIP.

Architecture

Next.js generation job:

```
for each annex:
  1. Gather { Field, Answer } pairs for this annex
  2. POST to http://pdf-service:8000/fill with JSON body
  3. Receive filled PDF bytes
  4. Save to /data/tenders/{tenderId}/output/{annexId}.pdf
Bundle all output PDFs into all.zip with `archiver`
Update job progress + status
```

Python service: `pdf-service/`

A standalone FastAPI app, separate from the Next.js codebase. Lives in a sibling directory in the same monorepo.

File layout:

```
pdf-service/
  pyproject.toml
  Dockerfile
  app/
    main.py          # FastAPI entrypoint
    schemas.py       # Pydantic models
    overlay.py        # PyMuPDF logic
  tests/
    test_overlay.py  # at minimum: smoke test on a sample page
```

Endpoints:

```
POST /fill
Request body (JSON):
{
  "pdf_base64": "<base64-encoded original PDF>",
  "fields": [
    {
      "page": 62,          # 1-indexed
      "bbox": { "x": 100, "y": 200, "width": 300, "height": 24 },
```

```

        "field_type": "SHORT_TEXT" | "LONG_TEXT" | "CHECKBOX" | "DATE" | "NUMBE
        "value": "<the user's answer as a string>",
        "font_size": 11                # optional, defaults to 11
    },
    ...
],
    "annex_pages": [62, 63, 64]      # optional: only output these pages. If omit
}
Response (200):
    application/pdf bytes (the filled PDF)
Errors:
    400 – invalid request schema
    422 – coordinate out of page bounds
    500 – PyMuPDF error

GET /health
    Returns { "ok": true } for Docker healthcheck

POST /debug
    Same body as /fill but returns a PDF with COLORED RECTANGLES overlaid at each b
    Used during development to verify coordinates from Claude Vision are correct.

```

How PyMuPDF handles Hebrew RTL (the reason we chose it)

PyMuPDF has native bidirectional text support via HarfBuzz (the same shaper Chrome uses). Three primary methods to use:

For short text in a known position (most fields):

```

import pymupdf
doc = pymupdf.open(stream=pdf_bytes, filetype="pdf")
page = doc[field.page - 1]
rect = pymupdf.Rect(x, y, x + width, y + height)
page.insert_htmlbox(
    rect,
    f'<div dir="rtl" style="font-family:Heebo; font-size:{font_size}px;">{html_es
)

```

`insert_htmlbox` was added in PyMuPDF v1.23.8. It uses HarfBuzz internally and correctly handles:

- Pure Hebrew text
- Hebrew + numbers ("123456789 .ר.ל") — numbers stay left-to-right in their proper position

- Hebrew + English mixed
- Word wrapping inside the rectangle
- Custom fonts via `<style>` with `@font-face`

For checkboxes:

```
# Draw a check mark inside the bbox
page.insert_text(
    point=pymupdf.Point(rect.x0 + 2, rect.y0 + rect.height * 0.8),
    text="✓",
    fontsize=font_size,
    color=(0, 0, 0)
)
```

For dates and numbers in known LTR contexts: Use `insert_text()` directly — no RTL handling needed.

Required font: Heebo

Bundle `Heebo-Regular.ttf` and `Heebo-Bold.ttf` in `pdf-service/app/fonts/`. Reference them from `insert_htmlbox` via:

```
css = """
@font-face {
    font-family: 'Heebo';
    src: url('/app/fonts/Heebo-Regular.ttf');
}
@font-face {
    font-family: 'Heebo';
    font-weight: bold;
    src: url('/app/fonts/Heebo-Bold.ttf');
}
"""

archive = pymupdf.Archive("/app/fonts")
page.insert_htmlbox(rect, html, css=css, archive=archive)
```

Coordinate space

Claude Vision returns coordinates in image pixel space. The Next.js app converts to PDF point space before calling `pdf-service`. PDF coordinate origin is bottom-left; PyMuPDF's `pymupdf.Rect(x0, y0, x1, y1)` uses **top-left** as origin (PyMuPDF normalizes this internally — verify in tests). Conversion utility lives in `lib/pdf/coords.ts` on the Next.js side.

Field type handling reference

Field type	How pdf-service draws it
SHORT_TEXT, ADDRESS, PHONE, EMAIL	<code>insert_htmlbox</code> with <code>dir="rtl"</code>
LONG_TEXT	<code>insert_htmlbox</code> , multi-line, wraps inside bbox
ID_NUMBER, COMPANY_ID	<code>insert_text</code> LTR (numbers always go LTR)
NUMBER, CURRENCY	<code>insert_text</code> LTR; for currency prepend ₪
DATE	<code>insert_text</code> in dd/mm/yyyy format LTR
CHECKBOX	<code>insert_text</code> "✓" at <code>bbox.x + 2</code>
RADIO	<code>insert_text</code> "●" at the matching option's bbox
SELECT	treat as SHORT_TEXT
SIGNATURE, STAMP, LAWYER_BLOCK	NEVER draw — these reach pdf-service only as a logical no-op or are filtered out before the call

Calling pdf-service from Next.js

Helper in `lib/pdf-service/client.ts`:

```
export async function fillPdf(pdfBytes: Buffer, fields: FieldFill[]): Promise<Buf
const response = await fetch(`${process.env.PDF_SERVICE_URL}/fill`, {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    pdf_base64: pdfBytes.toString('base64'),
    fields,
  }),
});
if (!response.ok) throw new Error(`pdf-service error: ${response.status}`);
return Buffer.from(await response.arrayBuffer());
}
```

Coordinate sanity check (debug mode)

The `/debug` endpoint on pdf-service returns the PDF with red rectangles at every field bbox

instead of text. Useful when calibrating coordinates from Claude Vision output during development. Expose it through Next.js as `GET /api/tenders/[id]/output/[annexId]?debug=1`.

Why not pdf-lib

`pdf-lib` would require: (1) manual Unicode bidi marks (`\u202B` , `\u202C`) for Hebrew+numbers, (2) hand-rolled word wrap using `font.widthOfTextAtSize()` , (3) manual right-alignment math, (4) edge-case handling for `5.11.11`/dates that consistently break. PyMuPDF's HarfBuzz integration handles all of this automatically. The Python sidecar costs ~50MB of container space and saves an estimated 1-2 weeks of edge-case debugging.

8. API routes (route handlers)

All routes are under `/api` . All require auth except `/api/auth/*` .

POST	<code>/api/auth/register</code>	- { email, password, name? } → { user }
POST	<code>/api/auth/login</code>	- handled by NextAuth
POST	<code>/api/auth/logout</code>	- handled by NextAuth
GET	<code>/api/tenders</code>	- list current user's tenders
POST	<code>/api/tenders</code>	- multipart upload OR { sourceUrl } → creates te
GET	<code>/api/tenders/[id]</code>	- full tender data (annexes, questions, mileston
DELETE	<code>/api/tenders/[id]</code>	- delete tender + files
GET	<code>/api/tenders/[id]/preview-page/[page]</code>	- returns rendered page image (cach
POST	<code>/api/tenders/[id]/answers</code>	- { questionId, value } → upsert an
POST	<code>/api/tenders/[id]/improve</code>	- { questionId, currentValue } → ca
POST	<code>/api/tenders/[id]/generate</code>	- kicks off PDF overlay job
GET	<code>/api/tenders/[id]/output/[annexId]</code>	- download single filled PDF
GET	<code>/api/tenders/[id]/output/zip</code>	- download all as ZIP
PATCH	<code>/api/tenders/[id]/milestones/[mid]</code>	- { done } → toggle milestone done
GET	<code>/api/jobs/[jobId]</code>	- job status snapshot
GET	<code>/api/jobs/[jobId]/stream</code>	- SSE for real-time updates
POST	<code>/api/scrape/mr-gov-il</code>	- { url } → { title, publisher, dea
GET	<code>/api/wizard/[tenderId]</code>	- { currentStep }
PATCH	<code>/api/wizard/[tenderId]</code>	- { currentStep } → save resume sta

Each route validates input with Zod. Each route checks `Tender.userId === session.user.id` to prevent cross-user access.

9. mr.gov.il scraper

The scraper accepts URLs of the form

`https://mr.gov.il/ilgstorefront/he/p/{tenderId}` .

Implementation:

- Use native `fetch` with a desktop User-Agent
- Parse with `cheerio`
- Extract:
 - Title from page heading
 - Publisher from metadata section (label "מפרסם")
 - Deadline from "מועד אחרון להגשה"
 - Tender number from "מספר הליך"
 - Contact from contact section
 - Attachment URLs from links containing `/p/attachment/`
- For each attachment, follow the link, sniff content type, save to `/data/tenders/{tenderId}/`
- Handle .zip attachments: unzip to a subdirectory

Attachments may be PDF, XLSX, DOCX, or ZIP. Treat ZIP as containing more of the above.

mr.gov.il has no rate limit issues for normal usage but be courteous: 1 second delay between attachment downloads.

10. AI prompts (specific text)

Provide these as exported constants in `lib/ai/prompts.ts` . They are critical to quality — iterate on them, log unexpected outputs, and version them.

Each prompt should:

- Specify language (Hebrew/English)
- Specify output format (JSON with schema)

- Include relevant context
- Include examples (few-shot) where helpful
- Have a clear "if unsure, return X" fallback

Example for the dedup prompt:

You are analyzing form fields from an Israeli government tender. The fields are i

INPUT: A JSON array of fields, each with: id, label (Hebrew), fieldType, annexCod

OUTPUT: A JSON array of questions. Each question has:

- id (generate as q1, q2, ...)
- category (one of: COMPANY_INFO, SIGNATORY_INFO, EXPERIENCE, DECLARATIONS, NARRA
- label (Hebrew, what the user will see)
- helpText (Hebrew, optional)
- fieldType (the most specific from the mapped fields)
- required (true if any mapped field is required)
- order (within category, starting at 1)
- fieldIds (array of input field ids that map to this question)

Rules:

- Group fields that ask for the same data, even if labels vary slightly:
 - * "שם החברה", "שם המציע", "שם התאגיד", "שם הספק" → one COMPANY_INFO question wi
- DO NOT include fields with fieldType SIGNATURE, STAMP, or LAWYER_BLOCK – these
- DO NOT invent fields. Only deduplicate what's given.
- The output's `fieldIds` arrays MUST collectively cover every input field except
- Sort questions logically: most fundamental data first (company name, ID) before

Return JSON only. No markdown fences, no explanations.

Provide similar carefully-engineered prompts for: file classification, annex segmentation, metadata extraction, field extraction (vision), milestone extraction, narrative improvement (OpenAI).

11. Auth implementation details

NextAuth v5 configuration:

- Credentials provider only
- JWT strategy (no DB sessions for v1 — simpler)
- Session expiry 30 days

- Password requirements: minimum 8 chars, at least 1 number, at least 1 letter (validate with Zod)
- Email validation: standard regex
- Hash with bcrypt cost factor 12
- Login throttle: max 5 failed attempts per email per hour (use a simple in-memory store for v1, document upgrading to Redis later)

Middleware protects all routes except `/login`, `/register`, and `/api/auth/*`.

12. Environment variables

Create a `.env.example`:

```
# App
APP_URL=http://localhost:3000
NODE_ENV=development

# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/tender_analyzer

# Auth
AUTH_SECRET=<generate with: openssl rand -base64 32>
AUTH_TRUST_HOST=true

# AI
ANTHROPIC_API_KEY=sk-ant-...
ANTHROPIC_MODEL=claude-sonnet-4-5
OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o-mini

# Storage
DATA_DIR=/app/data

# Job queue
PGBOSS_SCHEMA=pgboss

# PDF service
PDF_SERVICE_URL=http://pdf-service:8000
```

The `pdf-service/` directory has its own `.env.example`:

```
PORT=8000
LOG_LEVEL=info
FONT_DIR=/app/fonts
```

13. Deployment to Hetzner

Provide:

- `Dockerfile` for the Next.js app (multi-stage, output `standalone`)
- `pdf-service/Dockerfile` for the Python service (slim Python 3.12 base, install PyMuPDF + FastAPI + Uvicorn, copy fonts, expose port 8000, run with `uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 2`)
- `docker-compose.yml` with services: `app` (Next.js), `pdf-service` (Python), `postgres` , `caddy` (reverse proxy + auto Let's Encrypt)
- Internal Docker network so `app` can reach `pdf-service` at `http://pdf-service:8000`
- `pdf-service` is NOT exposed to the public internet — only the `app` container talks to it
- A README with deployment steps:
 1. SSH into Hetzner VPS
 2. Install Docker + Docker Compose
 3. Clone the repo
 4. Copy `.env.example` to `.env` and fill in
 5. `docker compose up -d --build`
 6. Run `prisma migrate deploy` inside the `app` container
 7. Configure DNS A record to point at the VPS IP
 8. Caddy will auto-fetch Let's Encrypt cert
- Health check endpoint at `/api/health` returning `{ ok: true, db: true, pdfService: true }` (the `pdfService` flag is set by pinging `http://pdf-service:8000/health`)
- Volumes for `/app/data` (uploaded files) and Postgres data
- Resource hints for Hetzner CX22 (2 vCPU, 4GB RAM): `app` gets 2GB, `pdf-service` gets 1GB, `Postgres` gets 512MB. PyMuPDF is memory-efficient but a 240-page PDF in flight

can briefly use ~200MB.

14. Code quality requirements

- TypeScript strict mode, no `any` (use `unknown` and `narrow`)
 - ESLint + Prettier, Tailwind class sorting plugin
 - All server-side code uses Zod for input validation
 - All AI responses validated with Zod before persistence
 - All file system writes go through a single utility module that enforces the data dir
 - All Claude/OpenAI calls go through a wrapper module with retry, timeout, and cost logging
 - No `console.log` in production paths — use `pino`
 - Component file structure: `app/`, `components/`, `lib/`, `lib/ai/`, `lib/pdf-service/` (client for the Python service), `lib/scrapper/`, and a sibling `pdf-service/` directory for the Python FastAPI app
 - Tests are NOT required for v1 but write the code in a way that's testable (pure functions for Hebrew text helpers, prompt builders, coordinate conversions)
-

15. Implementation order (suggested sprint plan)

Build in this order to keep the project demo-able at each milestone.

Sprint 0 (2-3 days) — DE-RISK SPIKE — DO THIS BEFORE ANYTHING ELSE

This is the most important step. Do NOT skip it. Before building any UI or auth, prove the hardest piece works:

- Set up just the `pdf-service/` Python project with FastAPI + PyMuPDF
- Bundle Heebo fonts
- Implement `/fill` endpoint with hardcoded test fields
- Take page 62 of the sample tender PDF (`samples/29-2026-3_מכרז.pdf` — נספח א'2 — "תצהיר בדבר אי תיאום מכרז") and overlay these test values:

- "שם החברה לדוגמה בע"מ" in the company name blank
- "123456789 .ת.ת" in the ID blank (mixed Hebrew + numbers)
- Today's date in the date blank
- "✓" in one of the checkbox fields
- Coordinates can be hardcoded in this spike — read them off the PDF manually using a tool like Apple Preview's "Tools → Show Inspector"
- Open the result in Adobe Acrobat AND on iPhone Preview AND in Chrome
- Print one copy

Pass criteria for Sprint 0:

- All Hebrew text reads right-to-left visually
- Mixed text "123456789 .ת.ת" renders correctly (the digits stay left-to-right within the Hebrew label, in the right position)
- Text fits inside the original blank lines, doesn't overlap the form's own text
- The PDF opens cleanly in all three viewers
- Print output looks acceptable

If Sprint 0 fails: stop. The whole project's premise depends on this working. Iterate on PyMuPDF parameters until pass, OR evaluate alternatives (PDFlib trial, Apyrse demo, fall back to HTML→PDF regenerate-the-form approach).

If Sprint 0 passes: continue to Sprint 1 with confidence.

Sprint 1 (week 1) — foundations

- Next.js + Tailwind + shadcn/ui + RTL setup
- Prisma schema + migrations + seed
- NextAuth credentials login/register
- Dashboard with empty state
- New tender modal (upload tab only, skip URL for now)
- Docker Compose with both `app` and `pdf-service` (already built in Sprint 0)

Sprint 2 (week 2) — extraction pipeline

- pg-boss job queue setup

- File classification pass
- Annex segmentation pass
- Tender metadata + milestone extraction
- Tender overview page (Summary tab populated)

Sprint 3 (week 3) — fields & dedup

- Field extraction with Claude vision
- Coordinate space conversion utilities
- Deduplication pass into Questions
- Forms tab populated
- Timeline tab populated

Sprint 4 (week 4) — wizard & answers

- Wizard skeleton (6 steps, navigation, progress bar)
- Auto-save answers
- Wizard resume state
- OpenAI narrative improvement
- Field-to-annex mapping helper text

Sprint 5 (week 5) — PDF overlay (production-ready)

- pdf-service hardening: handle all FieldType cases (LONG_TEXT wrapping, currency formatting, dates), proper error responses
- Next.js generation job: orchestrate calls to pdf-service per annex, save outputs, ZIP bundling
- Output page with preview, download, ZIP
- Debug mode for coordinate verification (red rectangles via `/debug` endpoint)
- End-to-end test on the real police tender

Sprint 6 (week 6) — polish

- mr.gov.il URL scraper
- Deadline notifications

- Error states throughout
 - Mobile responsiveness pass
 - Hetzner deployment + DNS
-

16. What is explicitly OUT of scope for v1

Do NOT build any of these. Mark with `// TODO v1.5` or `// TODO v2` if you encounter a tempting place to add them:

- Email-to-self of files (v1.5)
 - Email reminders for deadlines (v2)
 - Org/team accounts and shared tenders (v2)
 - Saved company profile / RAG over user's documents (v2)
 - Auto-discovery of new tenders from mr.gov.il (separate product)
 - Tender comparison (v2)
 - Direct submission to government portals (probably never)
 - קולות קוראים (different document type, separate product later)
 - Full handling of Excel as a fillable form — for v1, Excel is reference data only
 - Drafting completely new narrative answers from scratch (we only IMPROVE user's text, never fabricate)
 - Field-level comments/notes (v1.5)
 - Multi-language UI — Hebrew only
 - Admin panel
 - Analytics dashboards
 - Stripe / payments — pricing & monetization come after product validation
-

17. Sample test data

The user will provide two sample files in `samples/` :

- `samples/29-2026-3_מכרז.pdf` — the 240-page main PDF
- `samples/נספח_למכרז_29-2026_צפון.xlsx` — the 31-sheet equipment Excel

Use these for end-to-end testing. The expected behavior on these files:

- After upload, the system should detect ~11 annexes inside the PDF
- The Excel should be classified as `REFERENCE_DATA`, not as a fillable form
- Tender metadata: title contains "תחזוקה", publisher = "משטרת ישראל", deadline around 17/05/2026
- Milestones should include סיור קבלנים, deadline for clarifications, final submission deadline
- Field deduplication should produce roughly 20-30 questions across all annexes
- Generated PDFs should be byte-compatible with the original (same page count, same overall structure)

18. First steps for the implementing agent

Start with Sprint 0 (the spike). This is the most important step. Do not skip.

1. Read this entire spec
2. Set up only the `pdf-service/` Python project — FastAPI + PyMuPDF + Heebo fonts
3. Implement the `/fill` endpoint with hardcoded test data on page 62 of the sample tender
4. Verify Hebrew RTL output is correct in Adobe Acrobat, iPhone Preview, and Chrome
5. Print one copy
6. **Only after the spike passes, proceed to Sprint 1:**
 - Initialize the Next.js project with the listed dependencies
 - Set up Prisma + Postgres + initial migration
 - Build login/register + dashboard skeleton
 - Build the upload flow with a stub extraction job that just classifies files
7. Iterate from there following the sprint plan

When in doubt:

- Ask the user (Avital) for clarification
- Prefer simplicity over cleverness
- Optimize for ability to ship and iterate, not for “perfect” architecture
- The success metric is the police tender 29/2026 working end-to-end. Everything else is secondary.

When making decisions not covered here, document the decision in

`docs/decisions/{NNN}-{topic}.md` (ADR style) so future agents can understand why.

19. Notes on AI honesty & safety

- The tool produces filled tenders that the user will submit to government bodies. Errors have legal consequences.
 - The wizard’s review step (step 6) MUST clearly show every answer before generation, with the user explicitly clicking “צור מסמכים” to confirm.
 - The output page MUST include a final disclaimer: “המסמכים נוצרו אוטומטית. באחריותך. לבדוק את כל השדות לפני ההגשה.”
 - AI improvement of narrative text MUST preserve the user’s intent — the prompt to OpenAI explicitly says “improve grammar and professionalism only, do not invent facts or change meaning.”
 - Any field where AI’s confidence is low must be flagged in the UI (“נדרש אישור ידני”) rather than silently filled.
-

20. License notes

- **PyMuPDF is AGPL-3.** Acceptable for v1: this is an internal tool / early MVP / prototype. If the product is later commercialized as a SaaS sold to other companies, the SaaS source code would need to be released under AGPL OR a commercial PyMuPDF license must be purchased from Artifex (contact them directly for a quote — typically low-thousands €/year for small businesses). Document this in `docs/decisions/001-pymupdf-license.md` so the decision is explicit.
- All other dependencies in this spec are MIT, Apache 2, BSD, or similar permissive

licenses.

- Heebo, Noto Sans Hebrew, and similar fonts are SIL Open Font License — usable in any commercial product without restriction.

End of specification.